

A Preliminary Performance Analysis of LLM Inference on Edge Accelerators

Mayank Arya*, and Yogesh Simmhan

Department of Computational and Data Sciences, Indian Institute of Science, Bangalore 560012 India

Email: {mayankarya, simmhan}@iisc.ac.in

Abstract—Large Language Models (LLMs) have revolutionized machine learning by efficiently handling tasks like translation, text generation, summarization, and classification, making them ideal for applications such as virtual assistants and agentic workflows. However, their substantial computational requirements limit their wide adoption, particularly for smaller organizations lacking the computing infrastructure. Current LLM serving is done on the cloud, which imposes cost, latency and privacy concerns. Cloud–edge collaboration also struggles with network bottlenecks and model partitioning complexities. This work explores the potential for using edge accelerators, like Nvidia’s Jetson, for LLM inference. These compact devices have 100–1000s of GPU cores and 10+GB of shared GPU memory, allowing real-time edge inference close to the data source. We analyze and evaluate the efficiency of edge devices for LLM inference for different models and power modes, and make a case for tuning power modes to reduce power usage by ≈ 4 –6% while matching the throughput of the highest performance mode.

I. INTRODUCTION AND MOTIVATION

Large Language Models (LLMs) like OpenAI’s GPT-4, Meta’s Llama, etc. have significantly advanced Natural Language Processing through extensive training on large datasets. They excel in text generation, summarization, classification and human-like conversation. However, their remarkable capabilities come with high computing costs and memory demands. During LLM inference, the user provides a stream of *tokens* as a *prompt*. The model first processes these tokens in the *Prefill Stage*, followed by the *Decode Stage* where it iteratively generates output tokens based both on the user’s input and the prior output tokens produced.

Current LLM inference techniques use cloud computing which has drawbacks, such as network latency and high bandwidth utilization, costs and privacy concerns [1]. This paper explores the use of edge accelerators for efficient LLM inference (more precisely, their smaller variants or SLMs), bringing the models closer to the users. Edge devices like Nvidia’s Jetson Xavier AGX are compact and yet have a 512-core Volta GPU, an 8-core ARM CPU and 32GB of LPDDR4x memory shared between CPU and GPU, making them suitable for low-latency LLM inference. They also offer multiple power modes to control the number of active CPU and GPU cores, etc., enabling power–performance trade-offs [2].

LLMs are memory-intensive, primarily to store the model weights. High-end LLMs have billions or even trillions of parameters that need to be loaded into memory during inference.

* PhD Student Author

TABLE I: LLM Model used in experiments

LLM Model	# Parameters	Precision	Memory Req.
Llama-2-7b-hf	6.74 Billion	FP16	14GB
Llama-3.1-8b	8.03 Billion	BF16	16GB

TABLE II: Jetson Xavier power modes used in experiments

Power Mode	# Cores	Core Freq.	GPU Freq.
MaxN	8	2.2 GHz	1.4 GHz
A	8	1.2 GHz	1.4 GHz
B	8	0.6 GHz	1.4 GHz
C	4	2.2 GHz	1.4 GHz
D	4	1.2 GHz	1.4 GHz
E	4	0.6 GHz	1.4 GHz
F	2	2.2 GHz	1.4 GHz
G	2	1.2 GHz	1.4 GHz

However, quantized versions of LLMs reduce their memory footprint, trading-off between memory usage and accuracy, and enabling them to be run on edge accelerators. E.g., while Llama-3.1-8b with 8 billion parameters in full precision (32-bit) requires ≈ 32 GB of memory just for the weights, the quantized (16-bit) version we use needs only ≈ 16 GB.

At the same time, edge accelerators operate in power constrained environments. This requires us to find a power mode and inferencing batch size that reduces the energy footprint while not greatly affecting performance. In this poster, we provide an in-depth analysis of LLM inference on an Nvidia Jetson, measuring the throughput, power/energy consumption, and latency for diverse batch sizes for various power modes.

Related works such as EdgeShard [3], explore the use of Collaborative Edge Computing (CEC) to perform inference on full-precision Llama2 models; however, they lack baselines and leave room for further research.

II. EXPERIMENT SETUP

We evaluate the performance of the Nvidia Jetson Xavier AGX running Meta’s Llama2 and Llama-3.1 models (Table I) from Hugging Face. Our performance metrics are *throughput* and *accuracy*, assessed using the WikiText2 [4] and TinyStories [5] datasets for various power modes (Table II). The power modes vary the numbers of CPU cores and CPU frequency to study their impact on performance and power usage; we do not modify GPU frequency as it significantly affects performance, limiting the usefulness of inferencing.

Throughput in LLMs measures the total input and output tokens generated per second, indicating the model’s efficiency

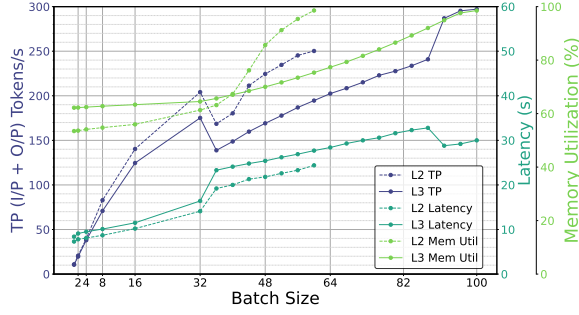


Fig. 1: Avg. Throughput (TP; input+output tokens/s), Avg. Batch Latency and Memory Usage for Llama2 (**L2**; dashed line) and Llama3.1 (**L3**; solid) running in *MaxN* power mode.

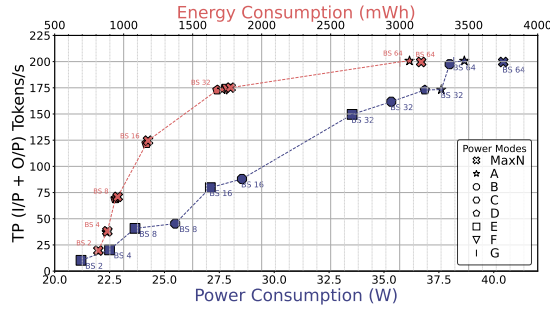


Fig. 2: Pareto plot of Throughput vs. Power and vs. Energy Usage for Llama3.1, for different power modes and *bs*. Label *BS* denotes the batch size and marker shape the power mode.

for inference tasks. We increase throughput by increasing the *batch size* (*bs*) for inference, feeding multiple requests to the model at once. Since we use text generation models, we use *perplexity* as the measure of accuracy. This measures how well the model predicts a given sequence of words (or tokens), i.e., the average uncertainty the model has when predicting the next token, given context of some previous tokens.

III. RESULTS

We evaluate *inference throughput* for $bs = 1 \dots 100$, beyond which the memory saturated. For each *bs*, we run 5 warm-up iterations to stabilize performance, followed by 10 measured runs. The final throughput is the average of these 10 runs. Each prompt has 10 input tokens and we generate 50 output tokens; so each request has 60 total tokens. We also report *batch inference latency* as the average time across the 10 stabilized runs, for each batch. We calculate *perplexity* for the entire WikiText2 and the first 10,000 rows of TinyStories dataset as single text sequences. The tokenized dataset is divided into overlapping chunks of 512 tokens, with each new chunk starting 500 tokens after the last to ensure 12 tokens of context is maintained across chunks. These are given below.

	WikiText2	TinyStories
Llama2	7.2235	4.0714
Llama3.1	8.2312	4.4537

Fig. 1 shows the inference throughput of Llama2 (L2) and Llama3.1 (L3) in **MaxN** power mode, which uses the all CPU cores and the maximum CPU/GPU/memory frequencies. This compares the Jetson's throughput, latency and memory utilization for different *bs*. As expected, both throughput and latency increase with *bs*. L2 achieves higher throughput due to fewer parameters but runs out of memory beyond $bs = 60$. The L3 model, which has improved memory optimizations, reaches $bs = 100$, with a peak throughput of 297 tok/s and latency of 30s for the batch. We observe a dip in throughput for both models after $bs = 32$, which we are investigating.

Subsequent experiments focused on Llama3.1, with *bs* range limited to $2 \dots 64$, in powers of 2. We profile the power and energy consumption for various power modes, and plot a Pareto front in Fig. 2 of Throughput vs. Energy (top X axis), and Throughput vs. Power (bottom). MAXN for $bs = 64$ is additionally shown to contrast with peak performance.

We make the following observations. Operating at power mode **A**, which uses all 8 cores but at a reduced frequency of 1.2GHz consumes 4.4% less power than **MaxN** which runs at 2.2GHz; but it achieves a throughput of 200 tok/s, marginally more than the 199 tok/s of MAXN. Using power mode **G** (2 cores@1.2GHz) reduces the power consumption by 5.6% over MAXN but has the identical throughput. This is because the CPU cores perform only lightweight tasks during LLM inference (kernel launches, preprocessing tasks such as tokenization, memory management between the CPU and GPU, etc.) compared to the heavy lifting done by the GPU (matrix operations, attention and model specific calculations). This also translates to energy benefits, with mode **A** having 2.7% lower energy consumption than MAXN, without any impact on throughput. This shows the benefits of intelligent power mode selection.

IV. CONCLUSION AND FUTURE WORK

LLM inference is a compute-intensive task, but most of the heavy lifting is done by the GPU. This allows us to limit the number of CPU cores required to be online and even lower their maximum frequency, leading to a decrease in total average power consumption without affecting the throughput.

Future work includes exploring the accuracy and performance trade-offs due to quantization of LLMs and parameter sizes, examining the effect of other power mode knobs such as memory frequency on the performance, and investigating LLM inference over distributed edge devices.

REFERENCES

- [1] B. Varghese *et al.*, "Challenges and opportunities in edge computing," in *IEEE SmartCloud*, 2016.
- [2] S.K. Prashanthi, S. A. Kesanapalli, and Y. Simmhan, "Characterizing the performance of accelerated jetson edge devices for training deep learning models," in *ACM SIGMETRICS*, 2023.
- [3] M. Zhang, J. Cao, X. Shen, and Z. Cui, "Edgeshard: Efficient llm inference via collaborative edge computing," Tech. Rep. arxiv:2405.14371, 2024.
- [4] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," Tech. Rep. arxiv:1609.07843, 2016.
- [5] R. Eldan and Y. Li, "Tinystories: How small can language models be and still speak coherent english?" Tech. Rep. arXiv:2305.07759, 2023.